

Roadmap de Implementación del HTTP Server en Rust

Metodología de Desarrollo

Este proyecto seguirá un enfoque **incremental y test-driven**, donde cada fase produce un sistema funcional y testeable. Cada milestone debe pasar todos los tests antes de continuar al siguiente.

Fases de Desarrollo

Fase 0: Configuración del Proyecto (Días 1-2)

0.1 Setup Inicial

Objetivo: Establecer la estructura base del proyecto y toolchain.

Tasks:

1. Crear nuevo proyecto Rust: `cargo new http-server --bin`
2. Configurar `Cargo.toml` con todas las dependencias necesarias
3. Crear estructura completa de directorios según la arquitectura
4. Configurar `.gitignore` y inicializar git repository
5. Setup de herramientas de desarrollo (rustfmt, clippy, tarpaulin)

Entregables:

- Proyecto compila sin warnings
- Tests básicos pasan (`cargo test`)
- Coverage tool configurado
- Scripts de build básicos funcionando

Verificación:

```
cargo check
cargo fmt -- --check
cargo clippy -- -D warnings
cargo test
```

0.2 Módulos Base

Objetivo: Implementar módulos fundamentales sin lógica compleja.

Tasks:

1. `src/error/types.rs` - Definir enum `ServerError` básico
2. `src/utils/logging.rs` - Setup básico de logging con `env_logger`
3. `src/config/mod.rs` - Estructura básica (sin CLI parsing aún)
4. Crear todos los archivos `mod.rs` vacíos con `TODO!()` macros

Entregables:

- Todos los módulos compilan (aunque con `todo!()`)
- Sistema de logging funcional
- Tipos de error básicos definidos

Tests Mínimos:

- Test que el logging produce output esperado
- Test que los tipos de error implementan `std::error::Error`

Fase 1: Parser HTTP y Servidor Básico (Días 3-5)**1.1 Parser HTTP/1.0**

Objetivo: Implementar parsing robusto de requests HTTP/1.0.

Tasks (en orden):

1. `src/server/request.rs`:
 - Estructura `HttpRequest` con campos básicos
 - Parser de request line (GET /path HTTP/1.0)
 - Parser de headers básico
 - Parser de query parameters
2. `src/server/response.rs`:
 - Estructura `HttpResponse` con builder pattern
 - Serialización a formato HTTP/1.0
 - Headers estándar (Content-Length, Content-Type)
3. Tests unitarios extensivos para casos edge

Entregables:

- Parser HTTP/1.0 completo y robusto
- Constructor de respuestas HTTP funcional
- 90%+ coverage en parsing HTTP

Tests Críticos:

```
#[test]
fn test_parse_simple_get_request() {
    let raw = "GET /fibonacci?num=10 HTTP/1.0\r\nHost: localhost\r\n\r\n";
    let req = HttpRequest::parse(raw).unwrap();
    assert_eq!(req.method, "GET");
    assert_eq!(req.path, "/fibonacci");
    assert_eq!(req.query_params.get("num"), Some(&"10".to_string()));
}

#[test]
fn test_malformed_request_returns_error() {
    let raw = "INVALID REQUEST\r\n";
```

```
    assert!(HttpRequest::parse(raw).is_err());  
}
```

1.2 Servidor TCP Básico

Objetivo: Servidor que acepta conexiones y responde con texto plano.

Tasks:

1. `src/server/connection.rs`:
 - Función para manejar una conexión TCP individual
 - Lectura con timeout
 - Manejo básico de errores de red
2. `src/server/http_server.rs`:
 - Estructura `HttpServer` básica
 - Loop de aceptación de conexiones
 - Delegación a handler de conexión
 - Graceful shutdown básico
3. `src/main.rs`:
 - Argument parsing básico para puerto
 - Inicialización y start del servidor

Entregables:

- Servidor que responde "Hello World" a cualquier request
- Manejo de múltiples conexiones concurrentes (un thread por conexión)
- Shutdown graceful con Ctrl+C

Test de Integración:

```
# Terminal 1  
cargo run -- --port 8080  
  
# Terminal 2  
curl http://localhost:8080/any-path  
# Debe responder: "Hello World"
```

Fase 2: Routing y Handlers Básicos (Días 6-8)

2.1 Sistema de Routing

Objetivo: Router que dirige requests a handlers específicos.

Tasks:

1. `src/server/router.rs`:
 - Trait `RequestHandler` para handlers
 - Estructura `Router` con tabla de rutas

- Matching de paths exactos
- Delegación a handler apropiado
- Respuesta 404 para rutas no encontradas

Entregables:

- Router funcional con trait-based handlers
- Registro dinámico de rutas

Tests:

```
#[test]
fn test_router_matches_exact_path() {
    let mut router = Router::new();
    router.register("/test", Box::new(TestHandler));

    let request = HttpRequest::new("GET", "/test", HashMap::new());
    let response = router.handle(&request);
    assert_eq!(response.status, 200);
}

#[test]
fn test_router_returns_404_for_unknown_path() {
    let router = Router::new();
    let request = HttpRequest::new("GET", "/unknown", HashMap::new());
    let response = router.handle(&request);
    assert_eq!(response.status, 404);
}
```

2.2 Handlers Básicos

Objetivo: Implementar todos los endpoints básicos especificados.

Tasks (implementar uno por vez, con tests):

1. `src/handlers/handler_traits.rs` - Traits y utilidades comunes
2. `src/handlers/basic.rs` - Implementar endpoints básicos:
 - `/help` (más simple)
 - `/timestamp`
 - `/reverse?text=...`
 - `/toupper?text=...`
 - `/hash?text=...`
 - `/fibonacci?num=...`
 - `/random?count=...&min=...&max=...`
 - `/createfile?name=...&content=...&repeat=...`
 - `/deletefile?name=...`
 - `/simulate?seconds=...&task=...`
 - `/sleep?seconds=...`

Estrategia: Implementar un endpoint por vez con su test correspondiente.

Entregables:

- Todos los endpoints básicos funcionando
- Validación de parámetros robusta
- Respuestas JSON según especificación
- Tests unitarios para cada endpoint

Test de Ejemplo:

```
#[test]
fn test_fibonacci_endpoint() {
    let mut params = HashMap::new();
    params.insert("num".to_string(), "10".to_string());

    let request = HttpRequest::new("GET", "/fibonacci", params);
    let response = fibonacci_handler(&request).unwrap();

    assert_eq!(response.status, 200);
    let body: serde_json::Value =
        serde_json::from_str(&response.body).unwrap();
    assert_eq!(body["result"], 55);
}
```

2.3 Integración con Servidor

Objetivo: Conectar todos los handlers con el servidor HTTP.

Tasks:

1. Registrar todos los handlers en el router
2. Integrar router con el servidor HTTP
3. Agregar headers de trazabilidad (X-Request-Id)
4. Implementar logging de requests

Entregables:

- Servidor HTTP completo con todos los endpoints básicos
- Logging de requests con IDs únicos
- Manejo de errores HTTP apropiado

Test de Integración Completo:

```
# Probar cada endpoint básico con curl
curl "http://localhost:8080/fibonacci?num=10"
curl "http://localhost:8080/reverse?text=hello"
curl "http://localhost:8080/help"
# etc.
```

Fase 3: Sistema de Workers (Días 9-12)

3.1 Worker Pool Básico

Objetivo: Sistema de workers thread-safe para tareas concurrentes.

Tasks:

1. `src/workers/worker_types.rs`:
 - Enum `TaskType` (Basic, CPU, IO)
 - Trait `Task` para tareas ejecutables
 - Estructura `TaskResult`
2. `src/workers/task_queue.rs`:
 - Cola thread-safe con channels
 - Métodos `push()`, `pop()`, `len()`
 - Backpressure cuando cola está llena
3. `src/workers/worker_pool.rs`:
 - Pool de N workers para un tipo específico
 - Workers que consumen tareas de la cola
 - Shutdown graceful de workers

Entregables:

- Worker pool funcional y thread-safe
- Sistema de colas con backpressure
- Tests de concurrencia básicos

Tests Críticos:

```
#[test]
fn test_worker_pool_processes_tasks() {
    let pool = WorkerPool::new(TaskType::Basic, 2);
    let task = TestTask::new("test data");

    pool.submit(task);
    thread::sleep(Duration::from_millis(100));

    assert_eq!(pool.completed_tasks(), 1);
}

#[test]
fn test_worker_pool_handles_concurrent_tasks() {
    let pool = WorkerPool::new(TaskType::Basic, 4);

    for i in 0..100 {
        pool.submit(TestTask::new(&format!("task-{}", i)));
    }

    // Wait for completion and verify no data races
    pool.wait_for_completion();
}
```

```
    assert_eq!(pool.completed_tasks(), 100);  
}
```

3.2 Worker Manager

Objetivo: Gestor centralizado de múltiples pools de workers.

Tasks:

1. `src/workers/worker_manager.rs`:
 - Estructura que gestiona múltiples pools
 - Configuración de workers por tipo de tarea
 - Routing automático de tareas al pool correcto
 - Métricas agregadas

Entregables:

- Manager centralizado de workers
- Configuración flexible de pools
- Métricas básicas por tipo de worker

3.3 Integración con Endpoints

Objetivo: Migrar endpoints existentes para usar workers.

Tasks:

1. Refactorizar handlers para crear `Task` objects
2. Enviar tareas a workers apropiados
3. Implementar timeouts para tareas
4. Manejo de errores de workers

Entregables:

- Todos los endpoints básicos usando workers
- Timeouts configurables
- Manejo robusto de errores de workers

Fase 4: Algoritmos CPU-Intensive (Días 13-16)

4.1 Algoritmos de Números Primos

Objetivo: Implementar algoritmos de primalidad y factorización.

Tasks:

1. `src/algorithms/prime.rs`:
 - Miller-Rabin primality test
 - Trial division factorization optimizada
 - Configuración de métodos (velocidad vs precisión)

2. `src/handlers/cpu_intensive.rs`:

- Handler `/isprime?n=...`
- Handler `/factor?n=...`
- Integración con worker system

Tests Críticos:

```
#[test]
fn test_miller_rabin_known_primes() {
    assert!(is_prime_miller_rabin(2));
    assert!(is_prime_miller_rabin(97));
    assert!(is_prime_miller_rabin(982451653)); // Known large prime
}

#[test]
fn test_factorization_correctness() {
    let factors = factorize(360);
    // 360 = 2^3 * 3^2 * 5^1
    assert_eq!(factors, vec![(2, 3), (3, 2), (5, 1)]);
}
```

Entregables:

- Algoritmos matemáticamente correctos
- Performance optimizado para números grandes
- Endpoints `/isprime` y `/factor` funcionando

4.2 Cálculo de π

Objetivo: Algoritmo iterativo para cálculo de π con precisión arbitraria.

Tasks:

1. `src/algorithms/pi_calculation.rs`:
 - Spigot algorithm o Chudnovsky
 - Control de precisión (número de dígitos)
 - Cancelación temprana para jobs largos
2. Handler `/pi?digits=...`

Tests:

```
#[test]
fn test_pi_calculation_first_10_digits() {
    let pi = calculate_pi(10);
    assert!(pi.starts_with("3.141592653"));
}
```

4.3 Mandelbrot y Matrices

Objetivo: Completar algoritmos CPU-intensive restantes.

Tasks:

1. `src/algorithms/mandelbrot.rs` - Generador de conjunto Mandelbrot
2. `src/algorithms/matrix_ops.rs` - Multiplicación de matrices grandes
3. Handlers correspondientes con validación

Entregables:

- Todos los endpoints CPU-intensive funcionando
- Tests de correctness para cada algoritmo
- Performance aceptable para tamaños especificados

Fase 5: Operaciones I/O Intensive (Días 17-20)

5.1 Operaciones Básicas de Archivos

Objetivo: Sistema base para manejo seguro de archivos.

Tasks:

1. `src/io_operations/file_ops.rs`:
 - Validación segura de paths (evitar directory traversal)
 - Operaciones streaming para archivos grandes
 - Manejo robusto de permisos
2. `src/algorithms/sorting.rs`:
 - External merge sort para archivos grandes
 - In-memory quicksort para archivos pequeños
3. Handler `/sortfile?name=...&algo=...`

Tests Críticos:

```
#[test]
fn test_sort_file_preserves_data() {
    let test_file = create_test_file_with_numbers(vec![3, 1, 4, 1, 5, 9]);
    sort_file(&test_file, SortAlgo::Merge).unwrap();

    let sorted_content = read_file_to_numbers(&test_file);
    assert_eq!(sorted_content, vec![1, 1, 3, 4, 5, 9]);
}

#[test]
fn test_sort_large_file() {
    let large_file = generate_file_with_size(50 * 1024 * 1024); // 50MB
    let start = Instant::now();
    sort_file(&large_file, SortAlgo::Merge).unwrap();
    let duration = start.elapsed();

    // Verify it completes in reasonable time
    assert!(duration < Duration::from_secs(30));
}
```

```
    assert!(file_is_sorted(&large_file));  
}
```

5.2 Procesamiento de Texto

Objetivo: Implementar operaciones tipo Unix (wc, grep).

Tasks:

1. `src/io_operations/file_processing.rs`:
 - `word_count()` compatible con `wc`
 - `grep_file()` con regex support
 - Processing streaming línea por línea
2. Handlers `/wordcount` y `/grep`

5.3 Compresión y Hashing

Objetivo: Completar operaciones I/O restantes.

Tasks:

1. `src/io_operations/compression.rs` - Compresión gzip/xz
2. `src/io_operations/hashing.rs` - SHA-256 de archivos
3. Handlers correspondientes

Entregables:

- Todos los endpoints I/O funcionando
- Soporte para archivos $\geq 50\text{MB}$
- Tests con archivos de diferentes tamaños

Fase 6: Sistema de Jobs Asíncronos (Días 21-26)

Esta es la fase más compleja del proyecto.

6.1 Tipos y Storage de Jobs

Objetivo: Infraestructura básica para jobs persistentes.

Tasks:

1. `src/jobs/job_types.rs`:
 - Estructura `Job` completa con metadatos
 - Enum `JobStatus` con todas las transiciones
 - Generación de UUIDs únicos
2. `src/jobs/job_storage.rs`:
 - Trait `JobStorage` para abstracción
 - Implementación con archivos temporales
 - Serialización/deserialización robusta

Tests:

```
#[test]
fn test_job_storage_persistence() {
    let storage = FileJobStorage::new("/tmp/jobs");
    let job = Job::new(TaskType::CPU, "isprime", params);

    storage.save_job(&job).unwrap();
    let loaded = storage.load_job(&job.id).unwrap();

    assert_eq!(job.id, loaded.id);
    assert_eq!(job.status, loaded.status);
}
```

6.2 Job Queue con Prioridades

Objetivo: Cola sofisticada que respeta prioridades.

Tasks:

1. `src/jobs/job_queue.rs`:
 - Binary heap para prioridades
 - FIFO dentro de misma prioridad
 - Thread-safe operations
 - Backpressure automático

Tests de Concurrencia:

```
#[test]
fn test_job_queue_respects_priorities() {
    let queue = JobQueue::new(100);

    queue.enqueue(job_low_priority());
    queue.enqueue(job_high_priority());
    queue.enqueue(job_normal_priority());

    assert_eq!(queue.dequeue().unwrap().priority, JobPriority::High);
    assert_eq!(queue.dequeue().unwrap().priority, JobPriority::Normal);
    assert_eq!(queue.dequeue().unwrap().priority, JobPriority::Low);
}
```

6.3 Job Manager Completo

Objetivo: Orquestador completo del sistema de jobs.

Tasks:

1. `src/jobs/job_manager.rs`:

- Gestión completa del ciclo de vida
- Integration con worker system existente
- Timeout management
- Progress tracking
- Cancelación de jobs

Features Críticas:

- Timeouts automáticos
- Progress reporting para jobs largos
- Cancelación segura
- Recovery después de restart

6.4 Job Endpoints

Objetivo: API completa para gestión de jobs.

Tasks:

1. `src/handlers/job_endpoints.rs`:
 - `/jobs/submit` - Creación de jobs
 - `/jobs/status` - Consulta de estado
 - `/jobs/result` - Obtención de resultados
 - `/jobs/cancel` - Cancelación
 - `/jobs/list` - Listado (debugging)

Tests de API:

```
#[test]
fn test_job_lifecycle_complete() {
    // Submit job
    let submit_response = submit_job("isprime", params);
    let job_id = submit_response.job_id;

    // Check status
    let status = get_job_status(&job_id);
    assert!(matches!(status.status, JobStatus::Queued |
JobStatus::Running));

    // Wait for completion
    wait_for_job_completion(&job_id);

    // Get result
    let result = get_job_result(&job_id);
    assert_eq!(result.status, JobStatus::Done);
    assert!(result.result.is_some());
}
```

Fase 7: Métricas y Observabilidad (Días 27-28)

7.1 Sistema de Métricas

Objetivo: Recolección comprensiva de métricas del sistema.

Tasks:

1. `src/utils/metrics.rs`:
 - `MetricsCollector` thread-safe
 - Histogramas para latencias
 - Cálculo de percentiles (p50, p95, p99)
 - Throughput y error rates
2. `src/handlers/metrics.rs`:
 - Handler `/metrics` completo según spec
 - Handler `/status` con información del servidor

7.2 Trazabilidad

Objetivo: Logging y tracing completo.

Tasks:

1. Implementar X-Request-Id en todos los responses
2. X-Worker-Pid para requests procesados por workers
3. Logging estructurado para análisis automático

Fase 8: Testing Comprensivo (Días 29-32)

8.1 Tests de Carga

Objetivo: Verificar comportamiento bajo carga alta.

Tasks:

1. `tests/load_tests.rs`:
 - Tests con N clientes concurrentes
 - Tests de saturación de workers
 - Detection de race conditions
 - Measurement de throughput y latencias

8.2 Tests de Integración Completos

Objetivo: Cobertura completa de todos los escenarios.

Tasks:

1. Tests end-to-end para cada endpoint
2. Tests de casos edge y límites
3. Tests de recovery y error handling
4. Tests de persistence de jobs

8.3 Benchmarks Performance

Objetivo: Benchmarks científicos para el informe.

Tasks:

1. **benches/** - Implementar todos los benchmarks
2. Medición de latencias p50/p95/p99
3. Análisis de escalabilidad
4. Comparación CPU vs I/O bound

Fase 9: Documentación y Entrega (Días 33-35)

9.1 Documentación Técnica

Tasks:

1. **README.md** completo con ejemplos
2. **docs/architecture.md** con diagramas
3. **docs/api_reference.md** completo
4. Documentación de código (rustdoc)

9.2 Informe Científico

Tasks:

1. Análisis de resultados de benchmarks
2. Comparación de algoritmos implementados
3. Análisis de escalabilidad y concurrencia
4. Conclusiones y trabajo futuro

Estrategias de Desarrollo

Test-Driven Development (TDD)

1. **Red:** Escribir test que falla
2. **Green:** Implementar código mínimo para pasar test
3. **Refactor:** Mejorar código manteniendo tests verdes

Integración Continua

Después de cada fase:

```
cargo test
cargo clippy -- -D warnings
cargo tarpaulin --out Html
cargo bench (cuando aplicable)
```

Debugging y Troubleshooting

- Usar **RUST_LOG=debug** para debugging detallado

- **cargo expand** para debuggear macros complejas
- **valgrind** o **heaptrack** para memory profiling si es necesario

Git Workflow

- Branch por feature: **feature/phase-1-http-parser**
- Commits atómicos con mensajes descriptivos
- PR/merge después de cada fase completada

Estimación de Tiempo

Tiempo Total Estimado: 35 días (7 semanas)

Distribución:

- Setup y fundamentos: 5 días (14%)
- HTTP Core: 8 días (23%)
- Workers y Algoritmos: 12 días (34%)
- Jobs System: 6 días (17%)
- Testing y Docs: 4 días (12%)

Factores de Riesgo:

- Fase 6 (Jobs) es la más compleja - considerar tiempo adicional
- Testing comprehensivo puede requerir más tiempo
- Performance tuning puede ser iterativo

Milestones de Validación

Milestone 1 (Fin Fase 2): Servidor HTTP básico funcional **Milestone 2** (Fin Fase 4): Todos los algoritmos CPU funcionando **Milestone 3** (Fin Fase 5): Todas las operaciones I/O funcionando

Milestone 4 (Fin Fase 6): Sistema de jobs completo **Milestone 5** (Fin Fase 8): Tests passing con 90%+ coverage

Cada milestone debe ser demostrable con curl/Postman y tener tests passing.